Chair of IT Security
Brandenburg University of Technology

Asya Mitseva, M.Sc.
Prof. Dr.-Ing. Andriy Panchenko

Winter Semester 2025/2026

Part V: Comparison of Detection Methods (Part 2) **Deadline: 21.01.2026**

Study Project: Adversarial Machine Learning
Use of Reverse Engineering for Anomaly Detection in Industrial Control Systems

Introduction

Industrial Control Systems (ICS) monitor and control critical infrastructures such as chemical plants, power systems, gas pipelines, and water systems. Typically, ICS consist of two macro areas, known as Information Technology (IT) and Operational Technology (OT). While IT is composed of standard computers interconnected via traditional communication protocols, OT includes special hardware and software for monitoring and controlling a given industrial facility. Overall, OT consists of *field*, *control*, and *supervisory* layers. The field layer includes different sensors and actuators, while the control layer is composed of a number of programmable logic controllers (PLCs). The sensors measure the current state of the underlying physical process, which is then reported to the PLCs. The PLCs implement a control logic used to evaluate the obtained sensor readings according to predefined or dynamically adjustable target values. Based on this evaluation, the PLCs may initiate specific commands to one or more actuators in the field layer aiming to adjust the current state of the physical process. In the supervisory layer, a Supervisory Control and Data Acquisition (SCADA) system performs a high-level visualization and control over the control and field layers and provides an overview of the current process state to the operator. While OT was mainly a standalone system in the past, nowadays it incorporates both decades-old devices and communication infrastructure, and new generation embedded devices with computing capabilities and Ethernet-based communication protocols, and is more often connected to the Internet. While the digitization of ICS makes the monitoring and the control of the physical process easier, it creates new attack surfaces against ICS.

A major line of research focused on the design and the development of robust intrusion detection techniques for the identification of cyber attacks in ICS. A particular challenge to be addressed is the usage of non-standardized protocols that are specific to manufactures or even to a particular type of device. Consequently, protocol specifications needed to process network data are not available and, thus, an in-depth analysis of the content of exchanged network traffic is difficult to accomplish.

The goal of this study project is to implement and evaluate different approaches that are able to extract expressive patterns, i.e., *features*, from network traffic collected in the OT area of a given ICS. We assume that the ICS operator utilizes proprietary binary protocols, i.e., the approaches to be developed cannot directly parse the content of transmitted network packets, and the defender relies on reverse engineering methods to derive only specific chunks from the observed network packets. The defender uses different machine learning (ML) methods with these chunks to identify a possible abnormal operation in ICS.

In the previous practical sheet, we continued with the evaluation of the performance of these methods using the features generated from your autoencoder. In the last part of the study project, we implement two more deep learning based classifiers and further compare the performance of the different detection methods created in the scope of this project.

Task 1: Ensemble classifier and Deep Learning based Classifiers

The goal of this task is to implement two deep learning based classifiers for binary classification and an ensemble classifier for both (*i*) one-class classification and (*ii*) multi-class classification by using the traditional machine learning methods implemented in the previous practical sheet. All classifiers should support those evaluation scenarios (i.e., types of k-fold cross-validation) from Practical sheet 3 (see Task 2, Item a)) for which their usage is meaningful. Both deep learning based classifiers are deep neural networks (DNNs) that can perform a classification of both complete network packets and unparsed physical readings (i.e., parts of the network packets that we extracted after using reverse engineering) by using the training and testing sets, created from the corresponding cross-validation. In particular, the first DNN should be a convolutional neural network (CNN) that takes as an input m x n complete network packets or unparsed physical readings, where m indicates the number of samples in a given class and n is the sequence of bytes of a single complete packet or the sequence of bytes of one or more unparsed physical readings, and produces a decision whether a single complete packet or a given set of unparsed physical readings contains (a given type of) attack, or not. In addition, your piece of code should satisfy the following requirements:

a) Your CNN should take as an input a predefined number of the sequence of bytes of a single complete packet or the unparsed physical readings whereas each sequence should consist of $n_p <= M$ bytes and M should be a parameter that is read from the command line.

b) Your CNN should contain six blocks in total before a prediction is printed out. Each of the first four blocks should contain two convolutional layers, batch normalization layers, activation functions, and dropout layers after each activation function. Then, the output from the fourth block should be passed to two fully-connected layers that will use the reduced data to classify the initial data to a given class. Last but not least, you should use Adam optimizer and categorical cross-entropy.

The second DNN should be a residual neural network (ResNet) that takes as an input m x n complete network packets or unparsed physical readings, where m indicates the number of samples in a given class and n is the sequence of bytes of a single complete packet or the unparsed physical readings, and produces a decision whether a single complete packet or a given set of unparsed physical readings contains (a given type of) attack, or not. In addition, your piece of code should satisfy the following requirements:

a) Your ResNet should take as an input a predefined number of the sequence of bytes of a single complete packet or the unparsed physical readings whereas each sequence should consist of $n_p <= M$ bytes and M should be a parameter that is read from the command line.

b) Your ResNet should contain 18 layers in total whereas these layers should be divided into four stages and each stage should consist of two convolutional blocks. Each convolutional block should contain two convolutional layers with batch normalization and ReLU non-linearity in-between. Then, your ResNet should pass the output from the convolutional layers to two fully-connected layers that will use the reduced data to classify the initial network packets to a given class. Last but not least, you should use Adam optimizer and categorical cross-entropy. Make sure that you identify and use the optimal hyper-parameters for your ResNet.

c) Beside the raw network packets, you should provide five basic statistical features to your ResNet. The first three features should include:

- the total number of bytes in a given sample (that is counted before trimming or padding this packet or unparsed physical readings for your ResNet),
- the number of occurrences of the first most often found byte in a network packet or unparsed physical readings,

the five features should be computed for each packet.

- the number of occurrences of the second most often found byte in a network packet or unparsed physical readings

The remaining two features should be proposed by you and should not repeat with the three features listed above. Elaborate why these features will be beneficial for your classification task. The user should be able to select from the command line whether to include the five statistical features to the input dataset when launching the classifier.

d) Additionally, you should write a piece of code that can perform hyperparameter tuning for both your CNN and your ResNet, e.g., optimization of the parameters needed for your convolutional layers, number of training epochs, etc.

Finally, your ensemble classifier should ensemble three traditional ML classifiers for both (i) one-class classification and (ii) multi-class classification using the following methods:

- *Random:* For each testing sample, you should randomly select one of the predictions of the three traditional ML classifiers included in the ensemble classifier.
- *Majority:* For each testing sample, you should select the majority of equal predictions from the three traditional ML classifiers included in the ensemble classifier.
- *All:* For each testing sample, you should output a positive prediction (i.e., an attack is present) only if all traditional ML classifiers included in the ensemble classifier predicted the presence of an attack in that sample.

The choice of an ensemble method should be done from the command line.

Task 2: Execution of Experiments

The goal of this task is to conduct an evaluation of your deep learning based classifiers (implemented in the previous task) and your ensemble classifier (implemented in the previous task) by using the features created through the autoencoder for each of the evaluation scenarios, defined in the third practical sheet. In particular, you should execute the following experiments:

a) For the second evaluation scenario, you should execute four experiments with your CNN classifier, in which you use either (i) entire network packets without applying the reverse engineering as input or (ii) parts of network packets for a sequence p of physical readings where $p = \{5, 10, 15\}$ after applying reverse engineering. For this item, you should use the raw bytes of the packets, not the features generated by the autoencoder. Make sure that for the detection method you identify and use the optimal parameters and compute the *precision* and *recall* for each of the experiments.

b) For the second evaluation scenario, you should execute four experiments with your ResNet classifier (without using the five basic statistical features), in which you use either (i) entire network packets without applying the reverse engineering as input or (ii) parts of network packets for a sequence p of physical readings where $p = \{5, 10, 15\}$ after applying reverse engineering. For this item, you should use the raw bytes of the packets, not the features generated by the autoencoder. Make sure that for the detection method you identify and use the optimal parameters and compute the *precision* and *recall* for each of the experiments.

c) For the second evaluation scenario, you should execute four experiments with your ResNet classifier (using the five basic statistical features), in which you use either (i) entire network packets without applying the reverse engineering as input or (ii) parts of network packets for a sequence p of physical readings where $p = \{5, 10, 15\}$ after applying reverse engineering. For this item, you should use the raw bytes of the packets, not the features generated by the autoencoder. Make sure that for the detection method you identify and use the optimal parameters and compute the *precision* and *recall* for each of the experiments.

d) For the third evaluation scenario, you should execute four experiments with your CNN classifier, in which you use either (*i*) entire network packets without applying the reverse engineering as input or (*ii*) parts of network packets for a sequence p of physical readings where $p = \{5, 10, 15\}$ after applying reverse engineering. For this item, you should use the raw bytes of the packets, not the features generated by the autoencoder. Make sure that for the detection method you identify and use the optimal parameters and compute the *precision* and *recall* for each of the experiments.

e) For the third evaluation scenario, you should execute four experiments with your ResNet classifier (without using the five basic statistical features), in which you use either (*i*) entire network packets without applying the reverse engineering as input or (*ii*) parts of network packets for a sequence p of physical readings where $p = \{5, 10, 15\}$ after applying reverse engineering. For this item, you should use the raw bytes of the packets, not the features generated by the autoencoder. Make sure that for the detection method you identify and use the optimal parameters and compute the *precision* and *recall* for each of the experiments.

f) For each of your evaluation scenarios, you should execute one experiment using the corresponding ensemble classifier with entire network packets without applying the reverse engineering as input. For this item, you should use the features generated by the autoencoder. Make sure that for each detection method you identify and use the optimal parameters and compute the *precision* and *recall* for each of the experiments.

g) For each of your evaluation scenarios, you should execute three experiments using the corresponding ensemble classifier with parts of network packets for a sequence p of physical readings where $p = \{5, 10, 15\}$ after applying reverse engineering as input. For this item, you should use the features generated by the autoencoder. Make sure that for each detection method you identify and use the optimal parameters and compute the *precision* and *recall* for each of the experiments.

Plot all obtained classification results in an appropriate way and submit your plots and the obtained classification results. Describe in details all takeaways that can be concluded after executing the experiments.

Task 3: Extension and Finalization of Your Toolbox

In Practical sheet 2, you created a toolbox that combined your piece of code prepared in Practical Sheet 1 and Practical Sheet 2. The goal of this task is to extend this toolbox by adding all remaining parts of your project implemented in the subsequent practical sheets in this toolbox. Your toolbox should provide several options that are read from the command line. The parameters that were already implemented in the previous practical sheet should be kept. In addition, the support of the parameter *-s <keyword>* should be extended to support the execution of all other parts of the project. Any already existing command line parameters can be kept and used in the toolbox as well.

Preparation for Q&A Session

Prepare yourself for a Q&A session of 40 minutes. During the Q&A session, you need to give a short overview of libraries, scripts or already existing tools that you have used. Furthermore, you need to make a short demo of your piece of code, explain its operation in detail, and show all classification results and plots created. Last but not least, you should be ready to answer spontaneous questions asked by the reviewer.